



Build a Security Monitoring System



Adeem Akhtar

```
{
  "Type": "Notification",
  "MessageId": "f3ac2dfb-124a-5b42-9a86-4f7889bbfb67",
  "TopicArn": "arn:aws:sns:us-east-2:370047118847:SecurityAlarms",
  "Subject": "ALARM: 'Secret is accessed' in US East (Ohio)",
  "Message": "{\n  \"AlarmName\": \"Secret is accessed\", \"AlarmDescription\": \"This alarm goes off whenever a secret in Secrets Manager is accessed.\", \"AWSAccountId\": \"370047118847\", \"AlarmConfigurationUpdatedTimestamp\": \"2026-05-17T10:17:18.724+0000\", \"NewStateValue\": \"ALARM\", \"NewStateReason\": \"Threshold Crossed: 1 out of the last 1 datapoints [5.0 (17/05/26 10:29:00)] was greater than or equal to the threshold (1.0) (minimum 1 datapoint for OK -> ALARM transition).\", \"StateChangeTime\": \"2026-05-17T10:34:25.642+0000\", \"Region\": \"US East (Ohio)\", \"AlarmArn\": \"arn:aws:cloudwatch:us-east-2:370047118847:alarm:Secret is accessed\", \"OldStateValue\": \"OK\", \"ActionExecutedAfterMuteWindow\": false, \"OKActions\": [], \"AlarmActions\": [\"arn:aws:sns:us-east-2:370047118847:SecurityAlarms\"], \"InsufficientDataActions\": [], \"Trigger\": {\n    \"MetricName\": \"Secret is accessed\", \"Namespace\": \"SecurityMetrics\", \"StatisticType\": \"Statistic\", \"Statistic\": \"SUM\", \"Unit\": null, \"Dimensions\": [], \"Period\": 300, \"EvaluationPeriods\": 1, \"DatapointsToAlarm\": 1, \"ComparisonOperator\": \"GreaterThanOrEqualToThreshold\", \"Threshold\": 1.0, \"TreatMissingData\": \"missing\", \"EvaluateLowSampleCountPercentile\": \"\"}\n  }\",
  "Timestamp": "2026-05-17T10:34:25.679Z",
  "SignatureVersion": "1",
  "Signature": "PuI9uHCuxHmITZTtcXKhc6WYKDFpgvz+xx+q32ip7oYWRv5LdKV7LuMLc2rdtMh6gO6OCFSQ/B9Lgx/1TEjNMJMdvpcDurZhahufma7fzPja2yYL0/Rg5PAKoeGjG1k0+IS0obPqJ52PFIIEQBZpA1vB+yMBR884oNrb/39PJn9I7z9rSsq4J6a/Ahw/8UlnbLgJRjJQ4TcCN/e47/RfaGclZJks1RkVZpG7oT2Ii vYxDqc/dk03Bs23WRjJZRp92JmsSyPQQh16VRNFWOpPoeLFRqAFDRILZRYfF85aCvdcJEU84TYKv7U9pzmp+cR1HwhbD5F13sP31SrRjxg==",
}
```



Introducing Today's Project!

In this project, I will demonstrate the following: 1. Set up AWS CloudTrail to track secret access events. 2. Use AWS CloudWatch to log access attempts and trigger notifications. 3. Create SNS alerts to get notified when your secrets are accessed.

Tools and concepts

Services I used were as follows: AWS Secrets Manager AWS CloudTrail Amazon CloudWatch Amazon SNS Amazon S3 AWS CLI Key concepts I learnt include: 1. Monitoring of the architecture, especially the secrets. 2. Triggering an alarm when specific patterns are detected. 3. Get notified as soon as possible about any unusual activity.

Project reflection

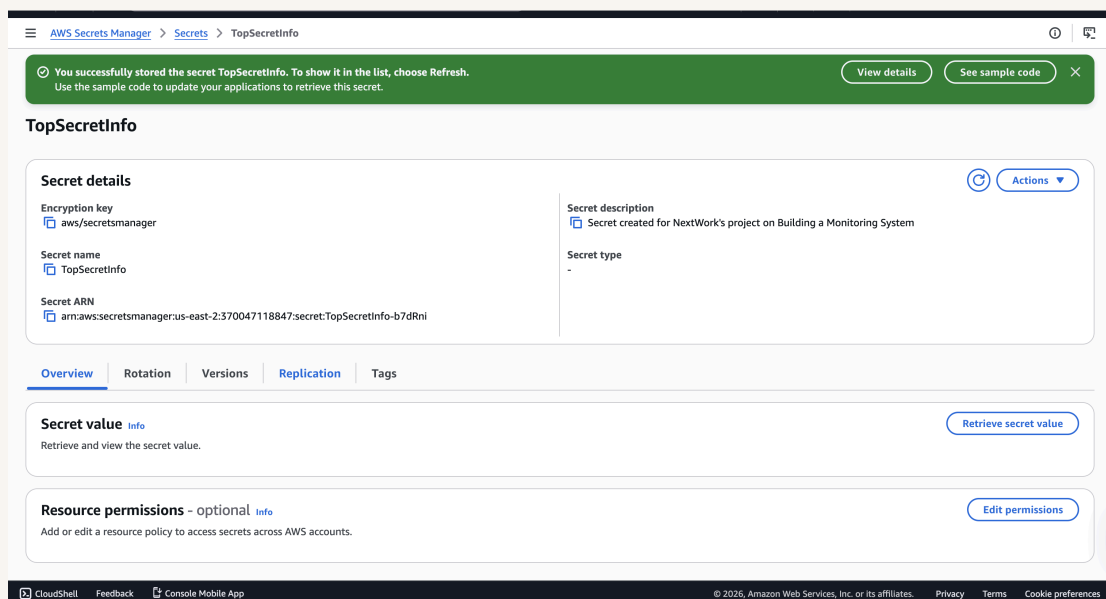
This project took me approximately 120 minutes. The most challenging part was troubleshooting of the email notification in SNS. It was most rewarding to figure out the subscription should be tested by sending the test emails.



Create a Secret

AWS Secrets Manager is an AWS service. We could use Secrets Manager to protect secrets, which are passwords, API keys, credentials and sensitive information. Instead of storing important credentials in your code or sharing them via email.

To set up for my project, I created a secret called "TopSecretInfo" that contains "I need 3 coffees a day to function".





Set Up CloudTrail

AWS CloudTrail is a monitoring service - think of it as an activity recorder throughout your AWS account. It documents every action taken, like who did what, when they did it, and where they did it from.

CloudTrail events include types like Management Event, Data Event, Insights Event and Network Activity Event. API Activities are "Read" and "Write"

Read vs Write Activity

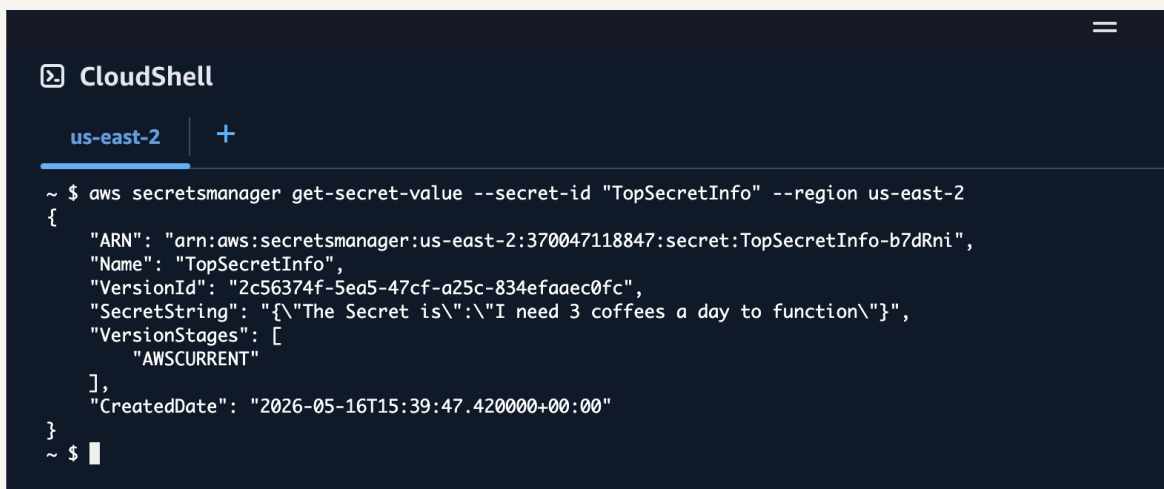
Read API activity happens when someone views but doesn't change anything. For example, listing your S3 buckets, describing EC2 instances. Write API activity occurs when changes happen - creating, deleting, modifying resources, or even retrieving the value of a secret.



Verifying CloudTrail

I retrieved the secret in two ways: First, through the AWS Management Console, and second, using AWS CLI.

To analyse my CloudTrail events, I visited the event history and found an event named "GetSecretValue". This tells me that the secret was accessed from my secrets manager.



```
CloudShell
us-east-2 +
~ $ aws secretsmanager get-secret-value --secret-id "TopSecretInfo" --region us-east-2
{
  "ARN": "arn:aws:secretsmanager:us-east-2:370047118847:secret:TopSecretInfo-b7dRni",
  "Name": "TopSecretInfo",
  "VersionId": "2c56374f-5ea5-47cf-a25c-834efaaec0fc",
  "SecretString": "{\"The Secret is\":\"I need 3 coffees a day to function\"}",
  "VersionStages": [
    "AWSCURRENT"
  ],
  "CreateDate": "2026-05-16T15:39:47.420000+00:00"
}
~ $
```



CloudWatch Metrics

Amazon CloudWatch Logs is a service that helps you consolidate logs from different AWS services, including CloudTrail. It's important for monitoring, visibility, troubleshooting, and analysis.

CloudTrail's Event History is useful for quick investigation about certain events, while CloudWatch Logs are better for long-term log retention and triggering automated actions.

A CloudWatch metric is a filter that automatically scans through your logs looking for specific patterns. When setting up a metric, the metric value represents the value that should be punched when the specific pattern is observed within the logs. The default value is used when keeping the track, even when the specific pattern is not observed.

Metric details

Metric namespace

Namespaces let you group similar metrics. [Learn more](#)

SecurityMetrics

Create new

Namespaces can be up to 255 characters long; all characters are valid except for colon(:) at the start of the name.

Metric name

Metric name identifies this metric, and must be unique within the namespace. [Learn more](#)

Secret is accessed

Metric name can be up to 255 characters long; all characters are valid except for colon(:), asterisk(*), dollar(\$), and space().

Metric value

Metric value is the value published to the metric name when a Filter Pattern match occurs.

1

Valid metric values are: floating point number (1, 99.9, etc.), numeric field identifiers (\$1, \$2, etc.), or named field identifiers (e.g. \$requestSize for delimited filter pattern or \$.status for JSON-based filter pattern - dollar (\$) or dollar dot (\$.) followed by alphanumeric and/or underscore (_) characters).

Default value – optional

The default value is published to the metric when the pattern does not match. If you leave this blank, no value is published when there is no match. [Learn more](#)

0

Unit – optional

Select a unit



CloudWatch Alarm

A CloudWatch alarm is a service that could be used to trigger another service, for example, SNS. I set my CloudWatch alarm threshold to 'static', 'greater/equal', 'than = 1', so the alarm will trigger when the "SecretIsAccessed" metric is greater than or equal to 1 in 5 minutes

I created an SNS topic along the way. An SNS topic is like a broadcast channel for your notifications. First, you create the channel (topic), then you invite subscribers, and finally, you send messages to the topic. SNS automatically delivers that message to all subscribers. My SNS topic is set up to my email for testing purposes.

AWS requires email confirmation because it verifies if the notification service is working properly and will send the notification to the subscriber. This helps prevent misconfiguration.



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:us-east-2:370047118847:SecurityAlarms:447c54b2-315a-4b59-a272-2b425467bcaa

If it was not your intention to subscribe, [click here to unsubscribe](#).



Troubleshooting Notification Errors

To test my monitoring system, I accessed my secrets from the AWS SecretsManager web interface and AWS CLI console. The results were "accessing my secrets".

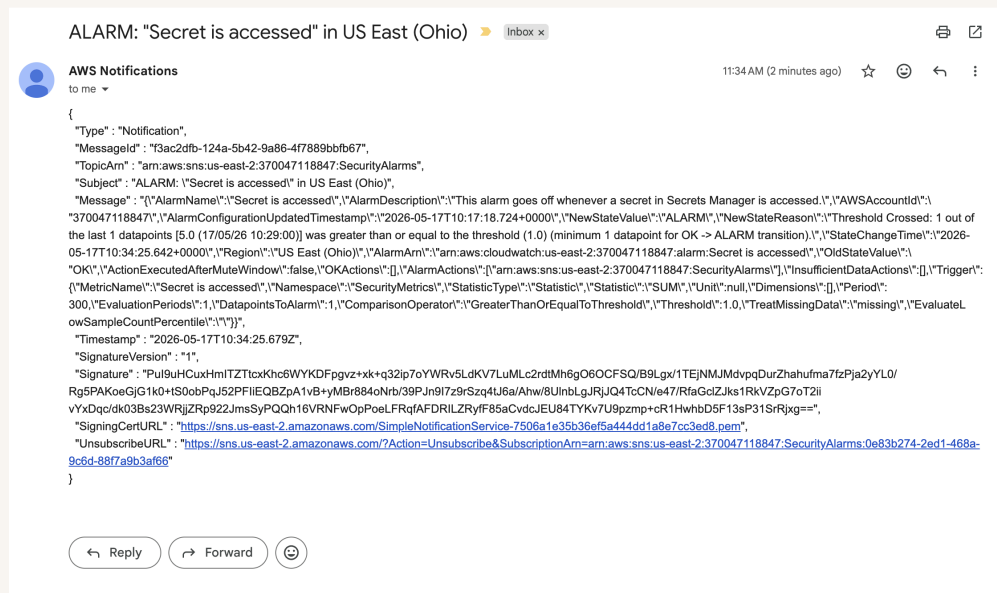
When troubleshooting the notification issues, I went through the following service reviews: > CloudTrail > Log Delivery > Metric Filter > Alarm Configuration > SNS Subscription

I initially didn't receive an email before because i did not confirm the subscription from my endpoint of test email The key solution was to open the email inbox and subscribe to the account.



Success!

To validate the system's working, when I checked the subscribed test email, I received a triggered alarm information saying that my secret had been accessed.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

